

Received 3 January 2026, accepted 12 January 2026, date of publication 19 January 2026, date of current version 23 January 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3655178

RESEARCH ARTICLE

Advanced Training Algorithms in Sigma-Delta Spiking YOLO for Energy-Efficient Object Detection on Neuromorphic Hardware

DONGYEONG KIM¹, GILHO KIM¹, MYEONG JUN KIM¹, SI YONG LEE¹,
AND YOON SEOK YANG², (Member, IEEE)

¹Department of Computer Science, Stony Brook University, Stony Brook, NY 11790, USA

²Department of Computer Science, The State University of New York Korea, Incheon 21985, Republic of Korea

Corresponding author: Yoon Seok Yang (yoonseok.yang@sunykorea.ac.kr)

ABSTRACT This paper explores methods for improving the training and inference performance of a Sigma-Delta spiking object detector by applying a series of training techniques and optimization strategies. Conventional CNN-based YOLO models, while effective in terms of accuracy, are generally unsuitable for deployment in low-power or real-time edge environments due to their computational demands. Spiking neural networks (SNNs), especially those employing Sigma-Delta neurons, offer a promising alternative with sparse, event-driven computation that reduces energy use. To this end, we enhance a Sigma-Delta spiking YOLOv3 framework by adapting several well-established training methods from both the SNN and CNN domains. These include Cosine-Annealing Learning-Rate Scheduling with Warm Restarts (SGDR), Multi-Gaussian surrogate gradients (MG-SGD), and a novel Epoch-Based Dynamic Surrogate Gradient Descent (ED-SGD) algorithm that we have devised. These additions are intended to improve both convergence behavior and overall model performance in spike-driven training. We evaluate the enhanced model using multiple real-world datasets, including BDD100K, Udacity Self-Driving, and SoccerNet. The experimental results show consistent improvements in detection accuracy and energy efficiency compared to both the conventional CNN-based YOLOv3 and the spiking baseline. Finally, we deploy the trained model on Intel's Loihi 2 neuromorphic platform, confirming its feasibility for real-time, low-power operation.

INDEX TERMS Advanced training algorithms, Sigma-Delta spiking neuron, spiking object detection, surrogate gradient algorithms, neuromorphic hardware.

I. INTRODUCTION

Object detection, an essential task in modern computer vision, has been dominated by convolutional neural networks (CNNs), particularly the YOLO family [1]. While models like YOLOv3 [2] balance accuracy and efficiency through streamlined backbones, their reliance on continuous, dense activation updates creates a significant energy bottleneck for power-constrained edge devices. This limitation has sparked interest in Spiking Neural Networks (SNNs) [3] as a low-power alternative. By utilizing asynchronous, event-driven spikes, SNNs drastically reduce power usage, a benefit

maximized by dedicated neuromorphic hardware such as Intel's Loihi [4], SpiNNaker [5], and Akida [6].

Among SNN architectures, Sigma-Delta neurons [7] are particularly promising because they encode the temporal derivative of input signals rather than absolute values, further minimizing spike rates. However, deploying Sigma-Delta SNNs for complex detection tasks is impeded by training difficulties: the non-differentiable nature of spike functions makes standard backpropagation infeasible, often leading to unstable gradients and poor accuracy on dense prediction tasks [8].

To address the optimization challenges of a Sigma-Delta spiking YOLOv3 detector—where sparse and discontinuous spike signals often yield weak or noisy gradients and

The associate editor coordinating the review of this manuscript and approving it for publication was Sotirios Goudos¹.

can cause early stagnation—we integrate complementary training mechanisms that explicitly balance exploration and stability. Specifically, we adopt Cosine-Annealing Learning-Rate Scheduling with Warm Restarts (SGDR) [10] to periodically re-increase the learning rate and escape suboptimal minima while maintaining smooth training dynamics, employ a MG-SGD [11] formulation to provide richer gradient support around the firing threshold for more effective gradient flow, and propose an ED-SGD method that modulates the surrogate-gradient magnitude across epochs, enabling more aggressive updates in early training and progressively more stable refinement in later stages. Together, these strategies form a unified training pipeline that improves convergence behavior and detection performance while remaining compatible with GPU-based training and subsequent neuromorphic deployment.

We evaluate this framework using the BDD100K [12], Udacity [13], and SoccerNet [14] datasets, comparing our enhanced model against a conventional CNN-based YOLOv3 (Darknet-53) [15] and a baseline Sigma-Delta SNN model from the Lava neuromorphic framework [16]. Finally, we validate the practical feasibility of the trained model on Intel's Loihi 2 neuromorphic hardware [17], confirming real-time operation and high energy efficiency.

The main contributions of this paper are as follows:

- We present a clear and reproducible end-to-end training pipeline for a Sigma-Delta spiking YOLOv3, integrating optimization scheduling and surrogate-gradient design to alleviate early stagnation and improve convergence behavior.
- We propose an ED-SGD method that dynamically scales the surrogate-gradient magnitude across epochs, enabling an effective exploration-to-stabilization transition during training.
- We validate the proposed training strategies across multiple datasets and demonstrate practical deployability by bridging GPU-based training with neuromorphic execution, highlighting energy-efficiency benefits on neuromorphic hardware.

II. RELATED WORK

In the domain of computer vision, object detection has emerged as a fundamental task. Deep convolutional neural networks (CNNs) are the architecture of choice for learning hierarchical spatial features [18], [19]. Among these, the YOLO family [1], [2] is influential, reframing detection as a single regression problem for fast, real-time detection [1]. However, this performance comes with a significant computational burden. YOLO-based models typically rely on deep CNN backbones composed of millions of parameters and require billions of floating-point operations per inference [20]. These constraints pose a serious challenge for applications targeting embedded, mobile, or neuromorphic platforms, where power, latency, and memory budgets are tightly constrained.

To address these limitations, researchers have turned to Spiking Neural Networks (SNNs) [3]. Inspired by the asynchronous and sparse communication patterns of biological neurons, SNNs offer a fundamentally different computational paradigm [21]. Unlike traditional neural networks that rely on continuous-valued activations and dense matrix operations [22], [23], SNNs transmit information through discrete spike events [3]. This enables computations only when necessary, leading to significantly lower energy consumption, particularly when deployed on neuromorphic hardware designed to exploit such temporal sparsity.

Among spiking models, Sigma-Delta neurons [7] are promising candidates. By encoding temporal changes in activation rather than absolute values, they minimize redundant computation [28] and align naturally with event-driven hardware paradigms [29]. While originally applied to classification [7], integrating Sigma-Delta neurons into object detection has become an emerging direction [31]. However, training remains difficult: the sparse, non-differentiable nature of spikes complicates gradient-based optimization [24], often leading to vanishing gradients and unstable convergence in dense prediction tasks [36]. Developing compatible detection models for platforms like Intel's Loihi is thus both practically and foundationally valuable [32], [33].

To mitigate these optimization instabilities, we draw inspiration from stage-wise learning, which aligns supervision with the learner's progress. Prior work shows that decomposing training into successive sub-stages can stabilize optimization and improve final performance (e.g., staged knowledge distillation and sub-stage graph formulations) [34], [35]. Analogously, we treat spiking optimization as a staged process across epochs: early epochs prioritize robust gradient flow, while later epochs emphasize fine-grained refinement. Concretely, we adopt a coarse-to-fine schedule by progressively shaping the surrogate-gradient signal across epochs.

Building on this stage-wise perspective, our work bridges the CNN and SNN domains to stabilize Sigma-Delta object detection. We move beyond static surrogate gradients [24], [38] by systematically integrating SGDR [10] and applying MG-SGD [11]. By harmonizing these methods with the epoch-wise surrogate schedule, we aim to regulate optimization dynamics throughout training, enhancing both convergence stability and final task performance.

III. PROPOSED WORK

A. NEURAL NETWORK ARCHITECTURE

The network that we used is built upon the CNN-based Tiny YOLOv3 architecture, integrated with the SDNN model within the Lava framework. The network architecture is composed of three primary components: the input layer, the backbone, and the detection head as shown in part a of Figure 1. It begins with an Input block that converts input frames into spike representations. In the Delta-encoding stage, frame-to-frame differences are computed and encoded as discrete event spikes. In the subsequent Sigma-decoding

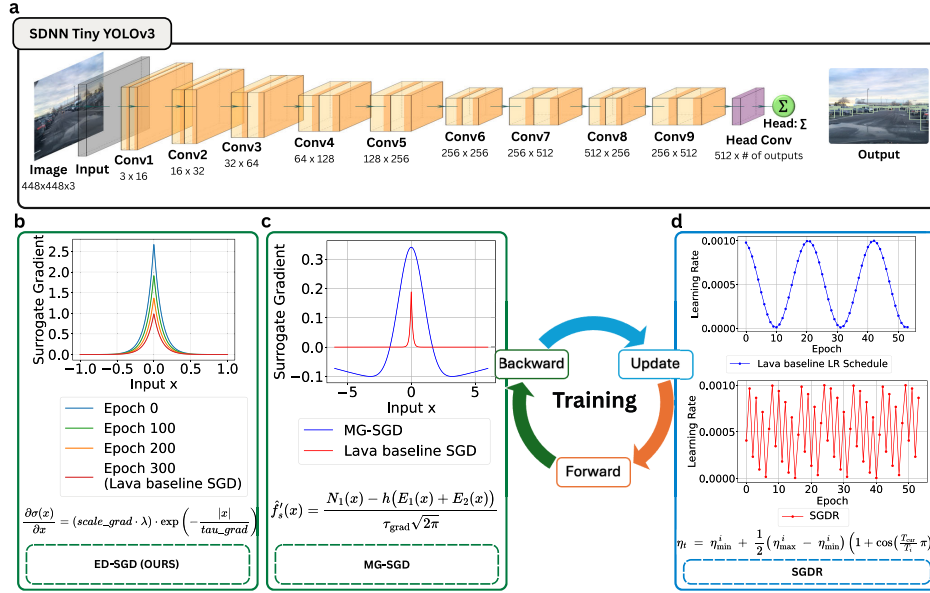


FIGURE 1. Sigma-Delta YOLO Network Architecture and Proposed Training Algorithm. Illustration of the proposed training strategies and model architecture for SDNN Tiny YOLOv3. (a) The overall architecture of our SDNN Tiny YOLOv3 model implemented in Lava. The input image of size 448x448x3 is processed through a stack of convolutional layers (Conv1–Conv9), followed by a spike-driven detection head. This architecture maintains the lightweight structure of Tiny YOLOv3 while supporting event-driven computation suitable for neuromorphic hardware. (b) ED-SGD, where the surrogate gradient descent function is adaptively modulated based on training epoch. This approach dynamically adjusts the spike gradient approximation over time. (c) MG-SGD function, which combines multiple Gaussian components to approximate the derivative of the spiking activation function. (d) SGDR is used during training to update the learning rate, avoiding local minima and discovering more effective weights.

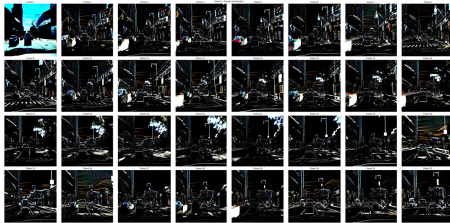


FIGURE 2. Visualization of the event generated from the input layer of the SDNN Tiny YOLOv3 network.

stage, these spikes are temporally integrated to reconstruct an approximate signal. This resulting event stream is then propagated to the next processing layer. Figure 2 demonstrates how the sigma-delta mechanism efficiently filters out static information and highlights motion-related changes. The backbone comprises nine sigma-delta convolutional layers (*slayer.block.sigma_delta.Conv*) with increasing channel widths from 3 to 512, using 3x3 and 1x1 kernels. The head consists of a 1x1 convolutional layer (*slayer.synapse.Conv*) projecting to the output dimensionality, followed by a Sigma dendritic nonlinearity. The architecture is fully spike-based and optimized for neuromorphic computation.

B. TRAINING ALGORITHM DESIGN

In this section, we first begin by outlining the training configuration of the SDNN Tiny YOLOv3 model provided by the Lava framework, which serves as our baseline [16], [41]. In the original implementation, only the BDD100K dataset

[12] is supported, and the model is trained for 200 epochs. The training pipeline applies only basic preprocessing—specifically, image resizing and normalization—without any additional data augmentation.

The Lava baseline training setting uses the Adam optimizer [42] with an initial learning rate of 1×10^{-4} , and a Lava baseline warmup-cosine learning rate schedule defined as:

$$\lambda(x) = \min\left(\frac{x}{E_{\text{warmup}}}, 1\right) \times \frac{1 + \cos\left(\frac{\pi x}{E_{\text{total}}}\right)}{2} \times (1 - \text{lrf}) + \text{lrf} \quad (1)$$

where:

- x is the number of current epoch
- E_{warmup} is the number of warmup epochs
- lrf is the learning rate factor

In addition, an Lava baseline exponential surrogate gradient [24] is used to replace the non-differentiable spike activation function with a smooth approximation:

$$\frac{\partial \sigma(x)}{\partial x} = \text{scale_grad} \times \exp\left(-\frac{|x|}{\tau_{\text{grad}}}\right) \quad (2)$$

where:

- x is the membrane potential.
- scale_grad controls the amplitude of the surrogate gradient.
- τ_{grad} controls the rate of exponential decay, acting as a relaxation parameter.

The `scale_grad` parameter acts as a multiplicative factor that controls the magnitude of the gradient. In the Lava baseline setting, this value is fixed throughout training, resulting in a constant gradient shape across all epochs. While effective, this approach does not adapt the learning signal over time, potentially limiting training dynamics, particularly in deeper or more temporally extended spiking networks.

In contrast to the Lava baseline setting, we propose a modified training scheme with different hyperparameter configurations, a new scheduling strategy, and a surrogate gradient descent function better suited for our task. Furthermore, we introduce a novel dynamic surrogate gradient formulation that improves the training mechanism for spiking neural networks.

1) OPTIMIZING HYPERPARAMETERS

We modify the Lava baseline hyperparameter configuration by increasing both the number of training epochs and the base learning rate. Specifically, we extend training to 300 epochs, allowing the network additional capacity to refine feature representations and improve generalization, especially critical in object detection tasks where spatial accuracy is essential. Moreover, we adopt a higher base learning rate of 1×10^{-3} instead of the Lava baseline configuration, which uses a conservative learning rate of 1×10^{-4} . This adjustment is motivated by the need to avoid early stagnation in training. A low learning rate can limit the model's ability to escape from poor local minima during the initial stages, particularly before the network has learned useful visual features. By enabling larger gradient updates throughout the training, the higher learning rate enables a more effective exploration of the loss landscape and accelerates the learning process.

2) UPDATING LEARNING RATE SCHEDULER

As part of our proposed training strategy, we replace the Lava baseline warmup-cosine scheduler with a Cosine-Annealing Learning-Rate Scheduling with Warm Restarts (SGDR) [10]. This change is motivated by the need to improve optimization dynamics in deep spiking neural networks, where non-convex loss landscapes and sparse gradient flow can lead to early stagnation or convergence to suboptimal solutions.

As illustrated in Fig. 1(d), the SGDR scheduler resolves these issues by periodically resetting the learning rate, allowing the optimizer to escape local minima and resume more aggressive exploration. Unlike monotonic decay schedules, SGDR introduces cycles of annealing, each followed by a warm restart that temporarily increases the learning rate before decaying again.

The learning rate at epoch t during the i -th cycle is computed as:

$$\eta_t = \eta_{\min}^i + \frac{1}{2}(\eta_{\max}^i - \eta_{\min}^i) \left(1 + \cos\left(\frac{T_{\text{cur}}}{T_i} \pi\right)\right) \quad (3)$$

where:

- η_t : learning rate at the current epoch t .

- $\eta_{\max}^i$ and $\eta_{\min}^i$: maximum and minimum learning rates for the i -th restart cycle.
- T_{cur} : number of epochs (or iterations) since the last restart.
- T_i : total length of the current cycle before the next restart.

At the beginning of each new cycle, the learning rate is reset to $\eta_{\max}^i$, then decays smoothly to $\eta_{\min}^i$ over the cycle interval T_i . This cyclical behavior helps avoid premature convergence, encourages better generalization, and provides regular opportunities for the optimizer to explore new regions of the parameter space [10].

SGDR also offers a qualitative stability benefit in spiking optimization: each warm restart acts as a controlled perturbation that can re-activate learning when the network enters a low-gradient regime, while the within-cycle cosine decay progressively reduces update magnitudes to support smoother late-stage refinement.

3) UPDATING SURROGATE GRADIENT DESCENT

a: MG-SGD [11]

To replace the Lava baseline exponential surrogate gradient descent, we adopt the MG-SGD proposed by Yin et al. [11], which improves learning stability and effectiveness in spiking neural networks. Inspired by ELU and Leaky ReLU activations, the MG-SGD consists of a central positive Gaussian and two negative Gaussians symmetrically placed on either side. As shown in Fig. 1(c), this shape provides broader gradient support beyond the spiking threshold, enabling nonzero gradients not only near the threshold but also in regions where conventional surrogates become flat.

The central component encourages spike generation when the membrane potential is close to the threshold, while the outer negative components help prevent inactive neurons from becoming permanently silent. As a result, the gradient function $\hat{f}'_s(x)$ offers smoother and broader support than standard Gaussian or exponential surrogates.

In our implementation, we tune the shape parameters h , s , and τ_{grad} to balance accuracy and sparsity. The specific values used and their empirical effects are reported in the *Experiments and Results* section.

$$N_1(x) = (1 + h) \exp\left(-\frac{x^2}{2\tau_{\text{grad}}^2}\right) \quad (4)$$

$$E_1(x) = \exp\left(-\frac{(x - \tau_{\text{grad}})^2}{2(s\tau_{\text{grad}})^2}\right) \quad (5)$$

$$E_2(x) = \exp\left(-\frac{(x + \tau_{\text{grad}})^2}{2(s\tau_{\text{grad}})^2}\right) \quad (6)$$

$$\hat{f}'_s(x) = \frac{N_1(x) - h(E_1(x) + E_2(x))}{\tau_{\text{grad}}\sqrt{2\pi}} \quad (7)$$

Qualitatively, the MG-SGD shape can be interpreted as widening the “effective learning band” around the spiking threshold: the central positive lobe promotes consistent credit assignment for near-threshold activity, whereas the

symmetric negative lobes provide a corrective pressure that discourages neurons from becoming permanently silent by maintaining nonzero gradients even when membrane potentials drift away from the threshold-support region. This broader and smoother gradient landscape reduces the likelihood of abrupt neuron dropout and helps sustain stable gradient flow across layers in deep spike-based detectors. As a result, training tends to exhibit fewer abrupt transitions between active and inactive spiking regimes, which is a common qualitative signature of instability in surrogate-gradient learning.

b: ED-SGD

To enhance the optimization process in SNNs, we introduce an ED-SGD strategy. The motivation behind this approach is to dynamically adjust the learning signal during training, promoting aggressive exploration in early epochs and stabilizing updates in later epochs to facilitate convergence.

This is achieved by modulating the magnitude of the surrogate gradient using a time-dependent scaling factor λ , defined as:

$$\lambda = \exp\left(\frac{E_{\text{total}} - E_{\text{current}}}{E_{\text{total}}}\right) \quad (8)$$

where E_{total} denotes the total number of training epochs, and E_{current} represents the current epoch index. The exponential form ensures a smooth, monotonic decay of λ from an initial high value toward 1 over the course of training. Qualitatively, the network experiences amplified gradients early on—when parameters are far from optimal and larger updates are beneficial—and more tempered gradients in the later stages, encouraging fine-grained adjustment. This behavior is summarized in Fig. 1(b), where λ decays over training to transition from larger early updates to more stable late-stage refinement.

We incorporate this scheduling into the Lava baseline surrogate gradient descent function. Specifically, the modified surrogate gradient descent function is expressed as:

$$\frac{\partial \sigma(x)}{\partial x} = (\text{scale_grad} \times \lambda) \times \exp\left(-\frac{|x|}{\tau_{\text{grad}}}\right) \quad (9)$$

Importantly, (9) changes not the local surrogate shape itself but the epoch-wise gain of the surrogate derivative. This preserves the desirable locality of exponential surrogates while qualitatively reducing late-epoch instability: even if many neurons remain near threshold, the overall update energy is moderated as training progresses, which helps prevent over-correction and supports smoother convergence behavior in dense prediction settings.

This adaptive formulation stands in contrast to the static surrogate gradient descent function used in the Lava baseline configuration, where the gradient magnitude remains fixed throughout training. By introducing flexibility into the learning signal, our method helps accelerate convergence in the early training phase while improving generalization and stability in the later epochs. Empirical results demonstrate

that this dynamic scaling strategy leads to faster convergence and higher final performance across a variety of tasks and datasets.

4) OVERALL PROPOSED ALGORITHM

The proposed SDNN Tiny YOLOv3 training algorithm (Algorithm 1) begins by initializing network parameters θ and configuring an *Adam* optimizer with learning rate 1×10^{-3} . A scheduler s is chosen—either the Lava baseline LR scheduler or SGDR LR scheduler (with $T_0 = 50$, $T_{\text{min}} = 10^{-6}$)—alongside a surrogate-gradient function g (Lava baseline SGD, MG-SGD, or ED-SGD). For each epoch $1 \leq \text{epoch} \leq E$, a decay factor $\lambda = \exp((E - \text{epoch})/E)$ is computed. The algorithm then iterates over each minibatch $(X, Y) \in \mathcal{D}$: after preprocessing, the network performs a forward pass $\hat{Y} = \text{net.forward}(X)$ to obtain predictions and spike counts, computes the YOLO loss $\mathcal{L} = \text{YOLO_Loss}(\hat{Y}, Y)$, and clears existing gradients. Depending on the selected surrogate, backpropagation uses one of Lava baseline SGD, MG-SGD or ED-SGD. After gradient clipping to ensure stability, *optimizer.step()* updates the parameters and *scheduler.step()* adjusts the learning rate. This design affords adaptive scheduling and flexible surrogate choices to enhance convergence and spiking network efficiency for object detection.

C. CUSTOM LOADER DESIGN

To support multiple datasets, the proposed loader operates without offline file reorganization, relying on an in-memory, virtual grouping mechanism. At initialization, the loader parses JSON index files (for training and validation), which enumerate each frame's filename and annotations. It then sorts the entire list of entries by filename to ensure temporal consistency and deterministic grouping. From this sorted list, images are partitioned into sequential groups based on a user-specified *images per group* parameter. When a group is requested by index, the loader selects a contiguous subsequence of frames of length L (the sequence length), loading each frame on-demand. If randomization is enabled, a uniform start offset is used; otherwise, the first L frames are selected. If a group contains fewer than L frames, the final frame is repeated to ensure a consistent sequence length. This on-the-fly logical grouping simplifies dataset management, ensures reproducible and temporally coherent batch sampling, minimizes I/O overhead, and integrates seamlessly with parallel data pipelines, making it highly scalable.

IV. EXPERIMENTS

A. DATASETS

We evaluate the Sigma Delta Neural Network (SDNN) object detection model on three diverse public datasets, chosen to cover urban driving scenarios, autonomous-vehicle test images, and dynamic non-urban scenes. Table 1 summarizes

Algorithm 1 Proposed SDNN Tiny YOLOv3 Training

```

1: Input: dataset  $\mathcal{D}$ , total epochs  $E$ ,
   Scheduler choice:

$$s \in \begin{cases} \text{Lava baseline LR scheduler,} \\ \text{SGDR LR scheduler} \end{cases}$$

   Surrogate choice:

$$g \in \begin{cases} \text{Lava baseline SGD,} \\ \text{MG-SGD,} \\ \text{ED-SGD} \end{cases}$$


2: Initialize network parameters  $\theta$ 
3: optimizer  $\leftarrow$  Adam( $\theta$ , lr= $10^{-3}$ )
4: if  $s$  = Lava baseline LR scheduler then
5:   scheduler  $\leftarrow$  Lava baseline LR scheduler
6: else if  $s$  = SGDR LR scheduler then
7:   scheduler  $\leftarrow$  SGDR LR scheduler( $T_0 = 50$ ,  $T_{\min} = 10^{-6}$ )
8: end if
9: for epoch = 1 to  $E$  do
10:   $\lambda \leftarrow \exp((E - \text{epoch})/E)$ 
11:  for each minibatch  $(X, Y)$  in  $\mathcal{D}$  do
12:    Preprocess  $(X, Y)$ 
13:     $(\hat{Y}, \text{counts}) \leftarrow \text{net.forward}(X)$ 
14:     $\mathcal{L} \leftarrow \text{YOLO\_Loss}(\hat{Y}, Y)$ 
15:    optimizer.zero_grad()
16:    if  $g$  = Lava baseline SGD then
17:      Backpropagate using Lava baseline SGD:

$$g'(u) = \exp(-|u|/\tau_{\text{grad}})$$

18:    else if  $g$  = MG-SGD then
19:      Backpropagate using MG-SGD:

$$g'(u) = \hat{f}'_s(u)$$

20:    else if  $g$  = ED-SGD then
21:      Backpropagate using ED-SGD:

$$g'(u) = (\text{scale\_grad} \times \lambda) \times \exp(-|u|/\tau_{\text{grad}})$$

22:    end if
23:     $\mathcal{L}.\text{backward}(g')$ 
24:    Clip gradients
    optimizer.step()
    scheduler.step()
25:  end for
26: end for

```

TABLE 1. Overview of datasets used for training and evaluation.

Dataset	Train Images	Val Images	# Classes
BDD100K	278,079	39,973	11
Udacity Self-Driving	22,350	7,450	4
Soccer Broadcast Frames	14,368	2,726	2

the number of training and validation images as well as the number of annotated classes for each dataset.

• BDD100K Dashcam Videos (BDD100K) [12]: A large-scale driving video dataset consisting of diverse urban environments, weather conditions, and lighting scenarios. Captured using vehicle-mounted dashcams, the data reflect real-world complexity in traffic scenes. Figure 3 a presents a sample frame from the BDD100K dataset with ground truth.

TABLE 2. Object category counts for each dataset.

BDD100K		Udacity		SoccerNet	
Category	Count	Category	Count	Category	Count
bicycle	32,884	biker	3,704	ball	13,394
bus	66,962	car	127,873	person	236,184
car	2,565,702	pedestrian	21,491		
motorcycle	13,379	truck	7,194		
other person	5,272				
other vehicle	22,032				
pedestrian	439,571				
rider	22,905				
train	1,900				
trailer	4,527				
truck	177,294				

TABLE 3. SDNN tiny YOLOv3 Network configuration for GPU.

layer	In Ch $\rightarrow$ Out Ch	Kernel Size	Stride	Padding
Input Block	—	—	—	—
Conv 1	3 $\rightarrow$ 16	3 $\times$ 3	2	1
Conv 2	16 $\rightarrow$ 32	3 $\times$ 3	2	1
Conv 3	32 $\rightarrow$ 64	3 $\times$ 3	2	1
Conv 4	64 $\rightarrow$ 128	3 $\times$ 3	2	1
Conv 5	128 $\rightarrow$ 256	3 $\times$ 3	1	1
Conv 6	256 $\rightarrow$ 256	3 $\times$ 3	2	1
Conv 7	256 $\rightarrow$ 512	3 $\times$ 3	1	1
Conv 8	512 $\rightarrow$ 256	1 $\times$ 1	1	0
Conv 9	256 $\rightarrow$ 512	3 $\times$ 3	1	1
Head Conv	512 $\rightarrow$ num_output	1 $\times$ 1	1	0
Head Sigma	—	—	—	—

Each frame has a resolution of 1280×720 pixels, allowing the SDNN's temporal encoding to track moving objects consistently across consecutive frames. Table 2 presents the annotated instance counts for each object category in the BDD100K dataset, illustrating its class distribution across urban driving scenes.

• Udacity Self-Driving Dataset (Udacity Self-Driving) [13]: High-resolution still images collected from autonomous vehicle tests. The dataset features clear views of roads, vehicles, and pedestrians under various driving conditions. Figure 3 b shows a sample frame from the Udacity Self-Driving dataset with ground truth. Each image in the dataset has a spatial resolution of 512×512 pixels. Table 2 outlines the number of labeled instances for each object class in the Udacity Self-Driving dataset, providing insight into its object frequency distribution.

• Soccer Broadcast Frames (SoccerNet) [14]: Single-frame extractions from televised soccer matches. The images contain fast-moving objects, motion blur, and non-standard perspectives, representing non-urban, dynamic visual contexts. Figure 3 c depicts a sample frame from the SoccerNet dataset with ground truth. Each image in the dataset has a spatial resolution of 1280×720 pixels. Table 2 summarizes the annotation counts for each object category in the SoccerNet dataset, highlighting the predominance of player and ball instances in sports broadcast frames.

B. NETWORK CONFIGURATION AND TRAINING

SDNN Tiny YOLOv3 was implemented using Intel's Lava framework (v0.10) [16], with Lava-dl (v0.60) [41] and SLAYER (v2.0) [25] for spiking neuron simulation and



FIGURE 3. Dataset Samples with Ground Truth (a) Sample data from BDD100K Dataset (b) Sample data from Udacity Dataset (c) Sample data from SoccerNet Dataset.

training. We used the Tiny YOLOv3 architecture with Sigma-Delta ReLU neurons and trained the model on an NVIDIA RTX 4080 GPU using PyTorch 2.3.1. To evaluate the relative performance of the proposed SDNN Tiny YOLOv3, we trained conventional convolutional neural networks (CNNs) as baselines. Specifically, we used Tiny YOLOv3 models implemented in Darknet, with two configurations: a large variant comprising 24 convolutional layers and a small variant with 12 layers. These CNN models were trained on the same dataset and under the same conditions—300 epochs, a learning rate of 0.001, and executed on the RTX 4080 GPU—to ensure a direct and fair comparison with the SDNN Tiny YOLOv3 models. The performance metrics from these CNN baselines were used as references to assess the detection accuracy, efficiency, and energy consumption characteristics of the spiking models.

Table 3 summarizes the layer-wise configuration of the SDNN Tiny YOLOv3 network deployed on the GPU. This includes the input and output channels, kernel size, stride, and padding for each convolutional layer, illustrating the architectural design employed for training the SDNN Tiny YOLOv3 on the PyTorch framework.

C. INTEL LOIHI2 CONFIGURATION AND INFERENCE

For neuromorphic deployment, we mapped the SDNN Tiny YOLOv3 object detection network onto Intel’s second-generation Loihi 2 [4] platform, employing five Oheo Gulch form-factor chips operating concurrently. The network was initially trained using PyTorch on a GPU, after which the learned weights were exported to a NetX-compatible HDF5 format, allowing direct translation into a computational graph suitable for execution on Loihi 2’s neurocores.

Notably, the spatial resolution of the input was modified by the architectural configuration of the first convolutional layer, which processed feature maps of size 224×224 . Thus, instead of conventional resizing pipelines, the input adaptation was inherently determined by the network’s receptive field as defined in the hardware layer parameters. This design choice ensured that the neuromorphic cores could accommodate the layer-wise activations within on-chip resources without exceeding memory or mapping constraints.

During inference, the input data underwent delta encoding and quantization before being transmitted to the Loihi 2 chips via dedicated injector and adapter processes. Inside the chip,

TABLE 4. Network configuration for Loihi 2.

Layer	W	H	C	Kernel Size	Stride	Padding
Conv 1	224	224	16	3×3	2	1
Conv 2	112	112	32	3×3	2	1
Conv 3	56	56	64	3×3	2	1
Conv 4	28	28	128	3×3	2	1
Conv 5	28	28	256	3×3	1	1
Conv 6	14	14	256	3×3	2	1
Conv 7	14	14	512	3×3	1	1
Conv 8	14	14	256	1×1	1	0
Conv 9	14	14	512	3×3	1	1
Conv 10	14	14	48	1×1	1	0

the computation proceeded entirely in a spike-based domain, exploiting the event-driven nature of the hardware to perform convolutional operations across successive layers. The resulting spike streams were then extracted and dequantized on the host side.

Finally, we utilized a dedicated *YOLOPredictor* module to transform these decoded outputs into bounding box coordinates and class probabilities, effectively reconstructing the object detection predictions. This hybrid processing approach leveraged Loihi 2’s neuromorphic advantages for core convolutional inference while relying on conventional host-based routines for the final decoding and non-maximum suppression steps. This architecture allowed us to maintain high throughput across multiple chips while achieving an energy-efficient inference pipeline optimized for the spiking domain.

Table 4 provides the detailed network configuration as mapped onto the Intel Loihi 2 Oheo Gulch platform. It lists the spatial dimensions (W, H), number of channels, kernel sizes, strides, and paddings for each layer, indicating how the network was adapted to fit the neuromorphic hardware constraints.

The complete process detailing how the models were converted and successfully mapped onto the Loihi 2 Oheo Gulch neuromorphic hardware is summarized by the resource utilization metrics presented in Tables 5. These tables break down the distribution of hardware resources across all network layers for deployments on Intel’s Loihi 2 chips. Each column indicates the proportion of the total usage accounted for by input axons, neuron groups, neurons, synapses, axonal mapping, and axon memory, respectively. The final column tallies the cumulative number of Loihi cores engaged.

TABLE 5. Per-layer resource utilization on Loihi 2 using different datasets.

(a) SoccerNet / Udacity Self-Driving (2 / 4 classes)								(b) BDD100K (11 classes)							
AxonIn	NeuronGr	Neurons	Synapses	AxonMap	AxonMem	Total	Cores	AxonIn	NeuronGr	Neurons	Synapses	AxonMap	AxonMem	Total	Cores
0.01%	12.50%	87.52%	72.00%	0.35%	0.09%	75.88%	32	0.01%	25.00%	65.64%	25.60%	16.80%	0.00%	47.38%	4
0.01%	12.50%	87.52%	54.40%	0.09%	0.35%	61.80%	14	0.01%	12.50%	87.52%	67.20%	0.09%	0.70%	72.32%	14
0.01%	12.50%	87.52%	57.60%	0.35%	0.35%	64.57%	32	0.01%	12.50%	87.52%	86.40%	0.35%	0.35%	87.61%	32
0.01%	12.50%	87.52%	57.60%	0.35%	2.80%	66.53%	16	0.01%	12.50%	87.52%	72.00%	0.35%	2.80%	78.05%	16
0.01%	12.50%	87.52%	64.80%	0.17%	0.17%	70.05%	56	0.01%	12.50%	87.52%	79.20%	0.17%	0.17%	81.57%	56
0.01%	12.50%	87.52%	32.40%	0.17%	0.35%	44.27%	28	0.01%	12.50%	87.52%	39.60%	0.17%	0.35%	50.03%	28
0.01%	12.50%	87.52%	7.20%	0.35%	0.17%	24.11%	56	0.01%	12.50%	87.52%	9.00%	0.35%	0.17%	25.55%	56
0.01%	12.50%	87.52%	2.70%	0.70%	0.35%	20.93%	112	0.01%	12.50%	87.52%	2.70%	0.70%	0.35%	20.93%	112
0.01%	12.50%	87.52%	0.34%	1.40%	0.70%	19.88%	224	0.01%	12.50%	87.52%	0.34%	1.40%	0.70%	19.88%	224
Total cores							572	Total cores							574

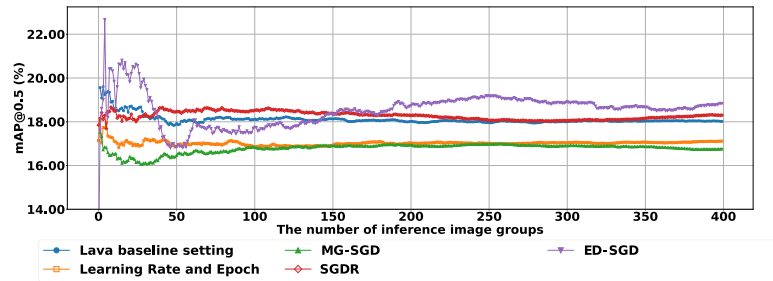
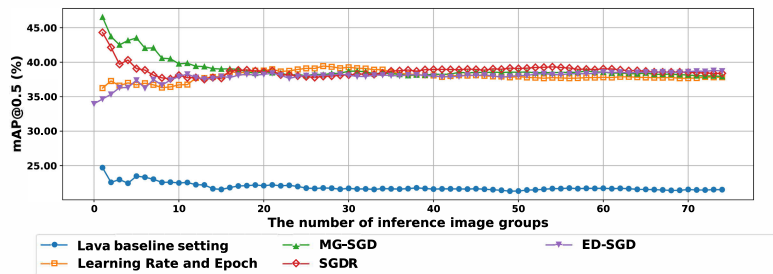
**FIGURE 4.** Comparison of mAP@0.5 performance on the BDD100K validation set using models trained with five different learning strategies. Each curve shows the inference accuracy of a trained model (.pt) across sequentially grouped test images. Among all methods, the model trained with ED-SGD consistently outperforms others, with an average of 18.53% accuracy with improved stability.**FIGURE 5.** Comparison of mAP@0.5 performance on the Udacity Self-Driving validation set using models trained with five different learning strategies. Each curve shows the inference accuracy of a trained model (.pt) across sequentially grouped test images. The models trained with Learning Rate and Epoch maintain significantly higher accuracy (average of 38.87%) than the Lava baseline, which remains average of 21.41%.

Table 5 (a) presents results obtained from the SoccerNet and Udacity Self-Driving datasets with 2-class and 4-class configurations, respectively. Table 5 (b) shows results obtained from the BDD100K dataset with 11 object classes. The increased class diversity and visual complexity in BDD100K require more input and output channels, thereby increasing input axon activity and resulting in a slightly higher total core count compared to the smaller-scale datasets.

V. RESULTS AND ANALYSES

A. RESULT COMPARISON

1) TRAINING STRATEGIES ACCURACY COMPARISON

We evaluated five training strategies applied to the same SDNN Tiny YOLOv3 architecture: the Lava baseline

setting [16], [41], learning rate and epoch adjustment, SGDR LR scheduler [10], MG-SGD function [11], and our proposed ED-SGD function. We use COCO-style mAP@50 as the evaluation metric, which measures the mean average precision at an IoU threshold of 0.5.

BDD100K [12]: Among the three datasets, BDD100K posed the most challenging detection task due to its high object diversity and the prevalence of small and densely packed bounding boxes. Figure 4 illustrates this trend, showing that our method consistently maintains higher accuracy across inference image groups. In this setting, our proposed ED-SGD function achieved the highest detection accuracy with an mAP@0.5 of 18.53%, outperforming both the Lava baseline (18.13%) and all other strategies. Interestingly, while

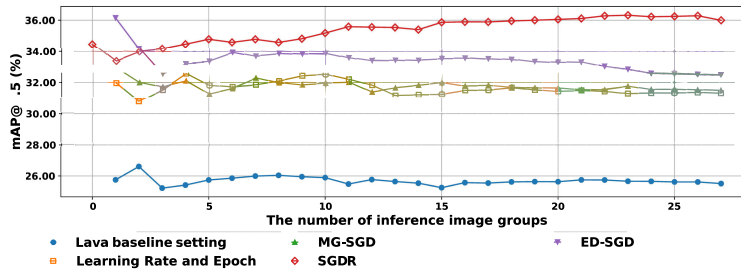


FIGURE 6. Comparison of mAP@0.5 performance on the SoccerNet validation set using models trained with five different learning strategies. Each curve shows the inference accuracy of a trained model (.pt) across sequentially grouped test images. The SGDR consistently achieves the highest accuracy, an average of 35.38%, while the Lava baseline setting maintains the lowest accuracy average of 25.8%.

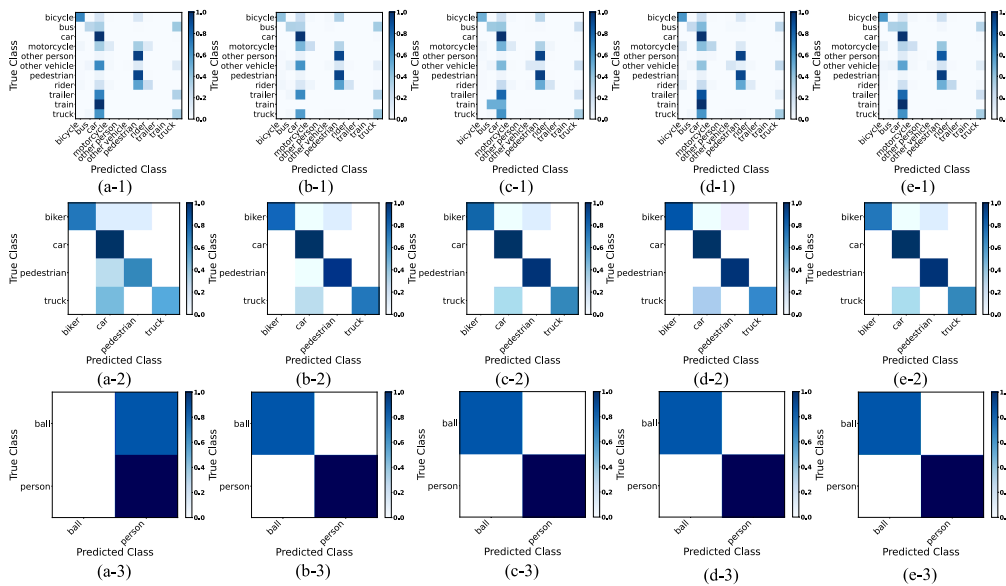


FIGURE 7. Comparison of confusion matrices on the (1) BDD100K, (2) Udacity Self-Driving and (3) SoccerNet validation set using models trained with five different learning strategies. (a) Applied Lava baseline training settings on the SDNN Tiny YOLOv3 network. (b) Applied updated learning rate and training epochs. (c) Applied SGDR. (d) Applied MG-SGD function. (e) Applied ED-SGD function. The confusion matrices demonstrate model performance in detecting two object classes in sports video scenes.

SGDR (18.28%) offered marginal improvement over the baseline, strategies such as learning rate and epoch tuning (17.02%) and MG-SGD function (16.79%) underperformed.

Figure 5 (1) across five ablation strategies on BDD100K reveals a clear progression: the Lava baseline training setting achieves moderate accuracy but suffers from notable class overlap; tuning the learning rate with extended epochs sharpens diagonals for major classes yet leaves rare categories under-detected; SGDR yields balanced, high accuracy with minimal confusion; ED-SGD function offers only partial gains for dominant classes; and the MG-SGD function produces the most uniformly strong diagonals, virtually eliminating misclassifications and substantially improving recall on underrepresented classes.

These results highlight the robustness of our method under complex and noisy training conditions.

Udacity Self-Driving [13]: The Udacity dataset contains fewer classes and more structured driving scenes, making it comparatively easier for object detection. As shown in Figure 5, performance gains from all improved strategies are visibly significant compared to the Lava baseline across all inference batches. Here, the best result was obtained using the learning rate and epoch tuning strategy (38.87%), slightly surpassing our proposed ED-SGD function (37.93%). However, our strategy still significantly improved upon the Lava baseline (21.41%) and yielded comparable performance to other advanced approaches such as SGDR (37.75%) and MG-SGD function (37.67%).

Figure 7 (2) compares confusion matrices from five training strategies on the Udacity Self-Driving dataset, covering four object classes. The Lava baseline training setting shows moderate accuracy with notable confusion between

similar classes such as biker and pedestrian. Applying a tuned learning rate and extended training significantly improves class separation, while SGDR further stabilizes training, yielding sharp diagonal patterns and minimal misclassification. ED-SGD function provides partial gains but retains some confusion. The MG-SGD function produces the clearest diagonal and lowest error, achieving the most balanced and precise detection.

Overall, this demonstrates that while simple hyperparameters such as learning rate and epoch tuning can achieve peak accuracy on well-structured datasets, our adaptive ED-SGD function remains a consistently robust contender.

SoccerNet [14]: The SoccerNet dataset presented a unique challenge with fast motion, non-urban scenes, and extreme scale variation. This performance trend is reflected in Figure 6, where both the SGDR and our proposed ED-SGD function consistently outperform the Lava baseline. In this domain, SGDR achieved the highest mAP@0.5 at 35.38%, while our ED-SGD function ranked second with 33.64%, still outperforming the Lava baseline (25.80%) by a large margin.

Figure 7 (3) illustrates how five ablation strategies progressively refine confusion matrix performance on the SoccerNet dataset: the Lava baseline training setting reliably detects players but only moderately identifies balls, often mistaking them for people; tuning the learning rate and extending training markedly sharpens both diagonals and reduces ball-to-player errors; SGDR further stabilizes updates, yielding high true-positive rates with virtually no off-diagonal errors; ED-SGD function brings only marginal improvements for the ball class while some confusion remains; and the MG-SGD function achieves the most uniform, intense diagonals, effectively eliminating misclassifications and maximizing detection accuracy for both balls and players.

Notably, all enhanced training strategies exceeded the Lava baseline, confirming the need for more sophisticated optimization techniques in spiking models under dynamic visual conditions.

Across all three datasets, the proposed ED-SGD function showed the most consistent improvement over the Lava baseline, achieving top-1 or top-2 results in every scenario. These results suggest that while data-specific strategies may excel in narrow cases, our ED-SGD function performs consistently across scenarios without requiring hand-tuning. In contrast, other strategies exhibited more dataset-specific variability, excelling in isolated cases, but lacking the generalization required for consistent performance across diverse visual contexts. This underscores the versatility and effectiveness of dynamically adapting the surrogate gradient descent function as training progresses, especially for temporally sparse spiking networks like SDNNs.

2) SDNN VS CNN ACCURACY COMPARISON

To evaluate the performance of spiking neural networks relative to conventional CNNs, we compared the proposed SDNN Tiny YOLOv3 with the Darknet-53 YOLOv3 Tiny model, consisting of 12 convolutional layers. As shown in

Table 6, both architectures contain approximately 3.5 million parameters, 3.474M for the Darknet-53 YOLOv3 Tiny with 12 layers and 3.488M for the SDNN Tiny YOLOv3, making the comparison fair in terms of model size and complexity.

Across all three datasets, the SDNN Tiny YOLOv3 consistently outperformed the Darknet-53 YOLOv3 Tiny with 12 layers. On the BDD100K dataset, the Darknet-53 YOLOv3 Tiny with 12 layers achieved a mAP@0.5 of just 3.64%, while the best-performing SDNN Tiny YOLOv3 with our proposed ED-SGD function reached 18.53%, representing a five times greater accuracy. This performance gap illustrates the SDNN's ability to extract and encode features from complex scenes with densely packed and small objects, leveraging its temporal dynamics.

On the Udacity Self-Driving dataset, which consists of clearer and less cluttered road scenes, the Darknet-53 YOLOv3 Tiny with 12 layers recorded 17.94%, whereas both SDNN Tiny YOLOv3 with proposed learning rate and epochs and ED-SGD function achieved 38.87% and 37.93%, again nearly doubling the accuracy. The improvement is particularly notable given the identical training budgets and architectural scale.

In the SoccerNet dataset, which consists of broadcast soccer footage with relatively static camera viewpoints and limited object classes, the Darknet-53 YOLOv3 Tiny with 12 layers achieved a mAP@0.5 of 32.88%. Despite this strong performance, the SDNN Tiny YOLOv3 still outperformed the Darknet-53 YOLOv3 Tiny with 12 layers, reaching 35.38% when trained with SGDR and 33.64% using our proposed ED-SGD function. Although the performance gap in this dataset was narrower compared to BDD100K and Udacity, the SDNN demonstrated competitive accuracy with similar parameters and event-driven computation. These results indicate that, even in scenarios with limited scene complexity and fewer object categories, well-trained spiking models can match or exceed traditional CNNs.

Overall, the SDNN demonstrates robust generalization and strong detection accuracy across diverse domains—urban driving, structured road scenes, and non-urban dynamic sports footage. As summarized in Table 6, these gains highlight the advantage of event-driven architectures in matching or outperforming CNNs of equivalent size, especially when optimized with SNNs exclusive strategies in the spiking domain.

3) ENERGY COMPARISON

To evaluate energy consumption, we performed repeated inference of a single frame for 5000 iterations under fixed clock and power settings for each platform. On the NVIDIA RTX 4080, we used the NVIDIA Management Library (NVML) to sample device power and computed energy by integrating power over the total runtime; latency was estimated by dividing the total runtime by the number of inferences. On Jetson Orin NX, power was measured using the onboard INA3221 sensor across the GPU/CPU/SOC rails, and energy was computed by integrating the measured

TABLE 6. Inference mAP@0.5 and Parameters for various architectures across datasets.

Dataset	Architecture	Param Count	Training Algorithm	Type	mAP@0.5 (Inference)
BDD100K	Darknet-53 YOLOv3 Tiny (24 layers)	8.269M	—	ANN	12.29%
	Darknet-53 YOLOv3 Tiny (12 layers)	3.474M	—	ANN	3.64%
	Lava SDNN Tiny YOLOv3	3.488M	—	SNN	18.13%
	Proposed SDNN Tiny YOLOv3	3.488M	Learning Rate and Epoch	SNN	17.02%
	Proposed SDNN Tiny YOLOv3	3.488M	SGDR	SNN	18.28%
	Proposed SDNN Tiny YOLOv3	3.488M	ED-SGD	SNN	18.53%
	Proposed SDNN Tiny YOLOv3	3.488M	MG-SGD	SNN	16.79%
Udacity Self-Driving	Darknet-53 YOLOv3 Tiny (24 layers)	8.269M	—	ANN	43.74%
	Darknet-53 YOLOv3 Tiny (12 layers)	3.474M	—	ANN	17.94%
	Lava SDNN Tiny YOLOv3	3.488M	—	SNN	21.41%
	Proposed SDNN Tiny YOLOv3	3.488M	Learning Rate and Epoch	SNN	38.87%
	Proposed SDNN Tiny YOLOv3	3.488M	SGDR	SNN	37.75%
	Proposed SDNN Tiny YOLOv3	3.488M	ED-SGD	SNN	37.93%
	Proposed SDNN Tiny YOLOv3	3.488M	MG-SGD	SNN	37.67%
SoccerNet	Darknet-53 YOLOv3 Tiny (24 layers)	8.269M	—	ANN	48.20%
	Darknet-53 YOLOv3 Tiny (12 layers)	3.474M	—	ANN	32.88%
	Lava SDNN Tiny YOLOv3	3.488M	—	SNN	25.80%
	Proposed SDNN Tiny YOLOv3	3.488M	Learning Rate and Epoch	SNN	29.56%
	Proposed SDNN Tiny YOLOv3	3.488M	SGDR	SNN	35.38%
	Proposed SDNN Tiny YOLOv3	3.488M	ED-SGD	SNN	33.64%
	Proposed SDNN Tiny YOLOv3	3.488M	MG-SGD	SNN	31.83%

TABLE 7. GPU results reflect dense CNN execution vs. timestep-based SNN simulation; Loihi 2 results reflect native event-driven execution.

Architecture	Type	Device	Total Energy (mJ)	Latency (ms)	Throughput (samples/s)	EDP (μ Js)	Param Count
Darknet-53 YOLOv3 Tiny (24 layer)	ANN	RTX 4080	1.55×10^5	2.20	3.12×10^2	4.95×10^5	8.269M
Darknet-53 YOLOv3 Tiny (12 layer)	ANN	RTX 4080	1.05×10^5	3.05	3.28×10^2	3.20×10^5	3.474M
SDNN Tiny YOLOv3	SNN	RTX 4080	1.98×10^5	1.20×10^1	4.91×10^1	3.95×10^6	3.488M
SDNN Tiny YOLOv3	SNN	Jetson Orin NX	2.30×10^4	3.59×10^2	2.78	8.26×10^6	3.488M
SDNN Tiny YOLOv3	SNN	Loihi 2	2.12×10^2	1.12×10^2	8.92	2.37×10^4	3.488M

system power over time. On Loihi 2, we used the Lava profiler to report per-inference energy directly on-chip. Note that the RTX 4080 measurements reflect conventional GPU execution (dense, frame-based kernels) for CNN baselines and time-stepped SNN simulation for SDNN inference; therefore, GPU results should be interpreted as software execution characteristics rather than neuromorphic event-driven efficiency.

On RTX 4080, the CNN baselines achieve better latency and EDP than the SDNN, because SDNN inference is executed as multi-timestep simulation in software rather than native event-driven hardware execution. As shown in Table 7, the SDNN Tiny YOLOv3 model on Intel's Loihi 2 neuromorphic chip demonstrates substantially lower energy and EDP than GPU- and edge-GPU-based executions. Specifically, Loihi 2 consumes 2.12×10^2 mJ per inference and achieves an EDP of 2.37×10^4 μ Js. This is approximately $495\times$ lower energy than Darknet-53 YOLOv3 Tiny (12 layers) on RTX 4080 (1.05×10^5 mJ) and about $731\times$ lower than Darknet-53 YOLOv3 (24 layers) on RTX 4080 (1.55×10^5 mJ). In terms of EDP, Loihi 2 is $13\times$ better than the 12-layer Darknet-53 variant (3.17×10^5 μ Js) and over $20\times$ better than the 24-layer Darknet-53 variant (4.95×10^5 μ Js).

In contrast, on RTX 4080 the SDNN Tiny YOLOv3 runs as a time-stepped spiking simulation within the Lava

software stack, which emulates neuromorphic execution on conventional GPU kernels and therefore cannot fully exploit hardware-level event-driven sparsity. As a result, SDNN inference exhibits higher latency and EDP than the CNN baselines on the same GPU. This outcome is expected because the GPU executes dense, frame-based kernels efficiently, whereas the Lava-based SDNN simulation incurs additional temporal unrolling and spike-processing overhead in software. Hence, the primary takeaway from Table 7 is not that SDNN inference is superior on GPUs, but that neuromorphic execution on Loihi 2 can convert spike sparsity into substantial energy and EDP gains. Therefore, the advantage of SDNN is not expected on a GPU, but emerges when deployed on neuromorphic hardware (Loihi 2), where event-driven execution yields substantially lower energy and EDP.

Compared to the Jetson Orin NX, which consumes 2.30×10^4 mJ with an EDP of 8.26×10^6 μ Js, Loihi 2 achieves roughly $108\times$ lower energy and more than $348\times$ better EDP. While Loihi 2 latency (1.12×10^2 ms) is higher than RTX 4080 (2.20–12.0 ms) and lower than Orin NX (3.59×10^2 ms), its throughput (8.92 samples/s) indicates practical performance for low-power, always-on perception workloads where energy efficiency is the dominant constraint. Overall, these results highlight that neuromorphic hardware is particularly advantageous when the SNN can be executed natively in an

event-driven manner, yielding large efficiency gains relative to conventional CNN inference on GPUs.

B. FUTURE WORK

Although our proposed framework achieves notable improvements in both detection accuracy and energy efficiency, several areas remain for further development. A current limitation of our work is that the overall mAP remains lower than state-of-the-art CNN-based detectors, reflecting the broader immaturity of SNN-based object detection. In addition, our analysis of training stability is mainly empirical; formal convergence guarantees for surrogate-gradient methods in SDNNs remain an open problem. A key direction is updating the underlying architecture to incorporate newer versions of YOLO, which offer enhanced feature extraction and detection performance. In parallel, we are exploring the integration of recent spike-based vision transformer techniques to introduce attention mechanisms and improve temporal representation within the spiking domain. To ensure practical applicability, we also plan to optimize the future framework for deployment on neuromorphic hardware such as Intel's Loihi 2, aiming to achieve real-time, low-power inference while maintaining the robustness of our model. These efforts will further elevate the model's performance and hardware compatibility in energy-constrained environments.

VI. CONCLUSION

We presented an enhanced training pipeline for SDNN Tiny YOLOv3 that improves convergence stability, detection accuracy, and energy efficiency. Across three real-world datasets, the proposed optimization components consistently outperform both CNN baselines and a Lava-based spiking baseline under comparable training budgets. Deployment on Intel Loihi 2 further confirms that the model can translate spike sparsity into substantial energy and EDP gains under practical latency/throughput constraints, supporting Sigma-Delta spiking detectors as a viable option for always-on, low-power perception.

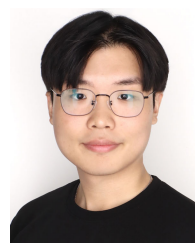
REFERENCES

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 779–788.
- [2] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," 2018, *arXiv:1804.02767*.
- [3] K. Roy, A. Jaiswal, and P. Panda, "Towards spike-based machine intelligence with neuromorphic computing," *Nature*, vol. 575, pp. 607–617, Nov. 2019.
- [4] M. Davies, N. Srinivasa, T.-H. Lin, G. N. Chinya, Y. Cao, S. H. Choday, G. D. Dimou, P. Joshi, N. Imam, S. Jain, Y. Liao, C.-K. Lin, A. Lines, R. Liu, D. Mathaikutty, S. McCoy, A. Paul, J. Tse, G. Venkataramanan, Y.-H. Weng, A. Wild, Y. Yang, and H. Wang, "Loihi: A neuromorphic manycore processor with on-chip learning," *IEEE Micro*, vol. 38, no. 1, pp. 82–99, Jan. 2018.
- [5] S. B. Furber, F. Galluppi, S. Temple, and L. A. Plana, "The SpiNNaker project," *Proc. IEEE*, vol. 102, no. 5, pp. 652–665, May 2014, doi: [10.1109/JPROC.2014.2304638](https://doi.org/10.1109/JPROC.2014.2304638).
- [6] *What is Akida Event Domain Neural Processor?*, BrainChip Inc., Sydney, NSW, Australia, 2020.
- [7] P. O'Connor and M. Welling, "Sigma delta quantized networks," 2016, *arXiv:1611.02024*.
- [8] Q. Su, Y. Chou, Y. Hu, J. Li, S. Mei, Z. Zhang, and G. Li, "Deep directly-trained spiking neural networks for object detection," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 6555–6565.
- [9] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, May 2015.
- [10] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," 2016, *arXiv:1608.03983*.
- [11] B. Yin, F. Corradi, and S. M. Bohté, "Accurate and efficient time-domain classification with adaptive spiking recurrent neural networks," *Nature Mach. Intell.*, vol. 3, no. 10, pp. 905–913, 2021.
- [12] F. Yu, H. Chen, X. Wang, W. Xian, Y. Chen, F. Liu, V. Madhavan, and T. Darrell, "BDD100K: A diverse driving dataset for heterogeneous multitask learning," 2018, *arXiv:1805.04687*.
- [13] *Self-Driving Car Dataset (Object Detection)*, Roboflow Contributors, Des Moines, LA, USA, 2024.
- [14] K. Moutselos and I. Maglogiannis, "Setting a baseline for long-shot real-time player and ball detection in soccer videos," in *Proc. 14th Int. Conf. Inf., Intell., Syst. Appl. (IISA)*, Jul. 2023, pp. 1–7.
- [15] A. Bochkovskiy, (2020), *Darknet: YOLOv4 Neural Networks*. [Online]. Available: <https://github.com/AlexeyAB/darknet>
- [16] *Lava: An Open-Source Software Framework for Neuromorphic Computing*, Intel Neuromorphic Computing Lab, Hillsboro, OR, USA, 2024.
- [17] G. Orchard, E. P. Frady, D. B. D. Rubin, S. Sanborn, S. B. Shrestha, F. T. Sommer, and M. Davies, "Efficient neuromorphic signal processing with loihi 2," in *Proc. IEEE Workshop Signal Process. Syst. (SiPS)*, Oct. 2021, pp. 254–259, doi: [10.1109/SiPS52927.2021.00053](https://doi.org/10.1109/SiPS52927.2021.00053).
- [18] M. D. Zeiler and R. Fergus, "Visualizing and understanding convolutional networks," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, Cham, Switzerland: Springer, Sep. 2014, pp. 818–833.
- [19] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 60, 2017, pp. 84–90.
- [20] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," 2015, *arXiv:1510.00149*.
- [21] W. Maass, "To spike or not to spike: That is the question," *Proc. IEEE*, vol. 103, no. 12, pp. 2219–2224, Dec. 2015.
- [22] D. Rumelhart, G. Hinton, and R. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, pp. 533–536, Sep. 1986.
- [23] W. Maass, "Networks of spiking neurons: The third generation of neural network models," *Neural Netw.*, vol. 10, no. 9, pp. 1659–1671, Dec. 1997.
- [24] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks," *IEEE Signal Process. Mag.*, vol. 36, no. 6, pp. 51–63, Nov. 2019.
- [25] S. B. Shrestha and G. Orchard, "SLAYER: Spike layer error reassignment in time," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 31, 2018, pp. 1412–1421.
- [26] Y. Wu, L. Deng, G. Li, J. Zhu, Y. Xie, and L. Shi, "Direct training for spiking neural networks: Faster, larger, better," in *Proc. AAAI Conf. Artif. Intell.*, Jul. 2019, vol. 33, no. 1, pp. 1311–1318.
- [27] H. H. Amin, "Automated adaptive threshold-based feature extraction and learning for spiking neural networks," *IEEE Access*, vol. 9, pp. 97366–97383, 2021.
- [28] S. Kundu, G. Datta, M. Pedram, and P. A. Beerel, "Spike-thrift: Towards energy-efficient deep spiking neural networks by limiting spiking activity via attention-guided compression," in *Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV)*, Jan. 2021, pp. 3952–3961.
- [29] D. Zambrano and S. M. Bohte, "Fast and efficient asynchronous neural computation with adapting spiking neural networks," 2016, *arXiv:1609.02053*.
- [30] A. Hasssan, J. Meng, and J.-S. Seo, "Spiking neural network with learnable threshold for event-based classification and object detection," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Yokohama, Japan, 2024, pp. 1–8, doi: [10.1109/IJCNN60899.2024.10650320](https://doi.org/10.1109/IJCNN60899.2024.10650320).
- [31] H. Zhang, Y. Li, B. He, X. Fan, Y. Wang, and Y. Zhang, "Direct training high-performance spiking neural networks for object recognition and detection," *Frontiers Neurosci.*, vol. 17, Aug. 2023, Art. no. 1229951.
- [32] S. Harbour, B. Sears, S. Schlager, M. Kinnison, J. Sublette, and A. Henderson, "Real-time vision-based control of SWaP-constrained flight system with Intel loihi 2," in *Proc. IEEE/AIAA 42nd Digit. Avionics Syst. Conf. (DASC)*, Oct. 2023, pp. 1–6.

- [33] J. E. Pedersen, S. Abreu, M. Jobst, G. Lenz, V. Fra, F. C. Bauer, D. R. Muir, P. Zhou, B. Vogginger, K. Heckel, G. Urgese, S. Shankar, T. C. Stewart, S. Sheik, and J. K. Eshraghian, "Neuromorphic intermediate representation: A unified instruction set for interoperable brain-inspired computing," *Nature Commun.*, vol. 15, no. 1, Sep. 2024.
- [34] C. Wang and Z. Tang, "The staged knowledge distillation in video classification: Harmonizing student progress by a complementary weakly supervised framework," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 34, no. 8, pp. 6646–6660, Aug. 2024, doi: [10.1109/TCSVT.2023.3294977](https://doi.org/10.1109/TCSVT.2023.3294977).
- [35] C. Wang, Z. Tang, and H. Xu, "WSSGCN: Wide sub-stage graph convolutional networks," *Neurocomputing*, vol. 602, Oct. 2024, Art. no. 128273, doi: [10.1016/j.neucom.2024.128273](https://doi.org/10.1016/j.neucom.2024.128273).
- [36] W. Fang, Z. Yu, T. Masquelier, Y. Chen, and Y. Tian, "Spike-based residual blocks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 21056–21069.
- [37] Y. Wu, L. Deng, G. Li, J. Zhu, and L. Shi, "Spatio-temporal backpropagation for training high-performance spiking neural networks," *Frontiers Neurosci.*, vol. 12, p. 331, May 2018.
- [38] M. Dampfhofer, T. Mesquida, A. Valentian, and L. Anghel, "Backpropagation-based learning techniques for deep spiking neural networks: A survey," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 9, pp. 11906–11921, Sep. 2024.
- [39] A. Shaban, S. S. Bezugam, and M. Suri, "An adaptive threshold neuron for recurrent spiking neural networks with nanodevice hardware implementation," *Nature Commun.*, vol. 12, no. 1, Jul. 2021, Art. no. 4234.
- [40] Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye, and D. Ren, "Distance-IoU loss: Faster and better learning for bounding box regression," in *Proc. AAAI Conf. Artif. Intell.*, Apr. 2020, vol. 34, no. 7, pp. 12993–13000.
- [41] *Lava-dl: A Deep Learning Library for Neuromorphic Computing*, Intel Neuromorphic Computing Lab, Hillsboro, OR, USA, 2024.
- [42] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.
- [43] T. J. Strain, L. J. McDaid, L. P. Maguire, and T. M. McGinnity, "A supervised STDP based training algorithm with dynamic threshold neurons," in *Proc. IEEE Int. Joint Conf. Neural Netw.*, Sep. 2006, pp. 3409–3414.
- [44] I. S. Isa, M. S. A. Rosli, U. K. Yusof, M. I. F. Marzuki, and S. N. Sulaiman, "Optimizing the hyperparameter tuning of YOLOv5 for underwater detection," *IEEE Access*, vol. 10, pp. 52818–52831, 2022.
- [45] C. Xu, Y. Liu, D. Chen, and Y. Yang, "Direct training via backpropagation for ultra-low-latency spiking neural networks with multi-threshold," *Symmetry*, vol. 14, no. 9, p. 1933, Sep. 2022.
- [46] F. C. Bauer, G. Lenz, S. Haghighatshoar, and S. Sheik, "EXODUS: Stable and efficient training of spiking neural networks," *Frontiers Neurosci.*, vol. 17, Feb. 2023, Art. no. 1110444.
- [47] Y. Kim, H. Park, A. Moitra, A. Bhattacharjee, Y. Venkatesha, and P. Panda, "Rate coding or direct coding: Which one is better for accurate, robust, and energy-efficient spiking neural networks?" in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, May 2022, pp. 71–75.
- [48] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal speed and accuracy of object detection," 2020, *arXiv:2004.10934*.
- [49] H. J. Chung, B. Kang, and Y. S. Yang, "N-DriverMotion: Driver motion learning and prediction using an event-based camera and directly trained spiking neural networks on loihi 2," *IEEE Open J. Veh. Technol.*, vol. 6, pp. 68–80, 2025, doi: [10.1109/OJVT.2024.3504481](https://doi.org/10.1109/OJVT.2024.3504481).
- [50] S. Snyder, S. R. Risbud, and M. Parsa, "Neuromorphic Bayesian optimization in lava," in *Proc. Int. Conf. Neuromorphic Syst. (ICONS)*, Aug. 2023, pp. 1–5.
- [51] N. Bodla, B. Singh, R. Chellappa, and L. S. Davis, "Soft-NMS: Improving object detection with one line code," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Aug. 2017, pp. 5561–5569.
- [52] D. Haase and M. Amthor, "Rethinking Depthwise separable convolutions: How intra-kernel correlations lead to improved MobileNets," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 14600–14609.
- [53] K. Kahatapitiya and R. Rodrigo, "Exploiting the redundancy in convolutional filters for parameter reduction," in *Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV)*, Waikoloa, HI, USA, 2021, pp. 1409–1419, doi: [10.1109/WACV48630.2021.00145](https://doi.org/10.1109/WACV48630.2021.00145).
- [54] F. Chollet, "Xception: Deep learning with Depthwise separable convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2017, pp. 1251–1258.
- [55] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
- [56] Q. Meng, M. Xiao, S. Yan, Y. Wang, Z. Lin, and Z. Q. Luo, "Towards memory- and time-efficient backpropagation for training spiking neural networks," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Jul. 2023, pp. 6166–6176.
- [57] C. Yu, L. Liu, G. Wang, E. Li, and A. Wang, "Advancing training efficiency of deep spiking neural networks through rate-based backpropagation," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, 2024, pp. 115786–115815.
- [58] M. Ghader, S. Reza Kheradpisheh, B. Farahani, and M. Fazlali, "Backpropagation-free spiking neural networks with the forward-forward algorithm," 2025, *arXiv:2502.20411*.
- [59] P. G. Cachi, S. Ventura, and K. J. Cios, "Improving spiking neural network performance with auxiliary learning," *Mach. Learn. Knowl. Extraction*, vol. 5, no. 3, pp. 1010–1022, 2023.
- [60] T. C. Wunderlich and C. Pehle, "Event-based backpropagation can compute exact gradients for spiking neural networks," *Sci. Rep.*, vol. 11, no. 1, Jun. 2021, Art. no. 12829.
- [61] C. Lee, S. Shakib Sarwar, P. Panda, G. Srinivasan, and K. Roy, "Enabling spike-based backpropagation for training deep neural network architectures," 2019, *arXiv:1903.06379*.
- [62] G. Shen, D. Zhao, and Y. Zeng, "Backpropagation with biologically plausible spatio-temporal adjustment for training deep spiking neural networks," 2021, *arXiv:2110.08858*.



DONGYEONG KIM is currently pursuing the Bachelor of Science degree in computer science with the Department of Computer Science, Stony Brook University, New York, USA. His research is centered on applying advanced neural network models to computer vision tasks. Specifically, focuses on using spiking neural networks to solve challenges using neuromorphic chips.



GILHO KIM is currently pursuing the dual B.S./M.S. degree in computer science with the Department of Computer Science, Stony Brook University, New York, USA. His primary research focuses on improving training strategies and energy efficiency for advanced neural networks. Particularly, centered on using spiking neural networks for challenging computer vision tasks.



MYEONG JUN KIM is currently pursuing the dual B.S./M.S. degree in computer science with the Department of Computer Science, Stony Brook University, New York, USA. His research interests include computer vision and vision-centric machine learning, with emphasis on perception tasks, such as object detection, recognition, and tracking.



SI YONG LEE received the M.S. degree in computer science and engineering from Chungnam National University, Daejeon, Republic of Korea, in 2024. He is currently pursuing the Ph.D. degree with the Department of Computer Science, Stony Brook University, New York, USA. His research interests include neuromorphic computing, algorithms and applications inspired by neural systems, event-based vision systems, and the design of AI accelerators.



YOON SEOK YANG (Member, IEEE) received the Ph.D. degree in electrical and computer engineering from Texas A&M University, College Station, TX, USA, in 2012. From 2000 to 2005, he was a Research Engineer with the Digital TV Laboratory of LG Electronics, Seoul, Republic of Korea. From 2012 to 2022, he was a Research Scientist with the Neuromorphic Computing Laboratory, Intel Labs, Santa Clara, CA, USA, where he focused on neuromorphic chip design.

From 2022 to 2023, he was a Silicon Engineer for Tensor Processing Units (TPU) Design with Google, Sunnyvale, CA, USA. Since 2023, he has been an Assistant Professor of computer science with The State University of New York Korea, Songdo, Republic of Korea. His research interests include neuromorphic computing, neuromorphic algorithms and applications, vision systems leveraging event-based sensors, accelerator design for artificial intelligence (AI), and machine learning-based image and signal processing.

• • •